

Git 常用命令汇总

以下是 Git 常用命令的分类整理，涵盖初始化、日常操作、分支管理、同步合并等核心场景，适合日常开发参考：

一、初始化与仓库配置

1. 初始化本地仓库

```
# 在当前目录初始化 Git 仓库（生成 .git 文件夹）  
git init
```

2. 克隆远程仓库到本地

```
# 克隆远程仓库（HTTPS 地址）  
git clone https://github.com/用户名/仓库名.git  
# 克隆远程仓库（SSH 地址，需配置 SSH 密钥）  
git clone git@github.com:用户名/仓库名.git
```

3. 配置用户信息（标识提交者）

```
# 全局配置（所有仓库生效）  
git config --global user.name "你的名字"  
git config --global user.email "你的邮箱"  
# 局部配置（仅当前仓库生效，去掉 --global 即可）  
git config user.name "你的名字"  
git config user.email "你的邮箱"  
# 查看配置  
git config --list
```

二、日常操作（文件暂存、提交、查看）

1. 查看文件状态

```
# 查看工作区与暂存区的文件状态（红色：未暂存；绿色：已暂存）
git status
# 简洁显示状态（适合快速浏览）
git status -s
```

2. 暂存文件（工作区 → 暂存区）

```
# 暂存单个文件
git add 文件名（如：git add lib/main.dart）
# 暂存当前目录所有文件（包括子目录）
git add .
# 暂存所有修改（包括删除的文件）
git add -A
```

3. 提交文件（暂存区 → 本地仓库）

```
# 提交暂存区文件，需填写提交信息（描述修改内容）
git commit -m "提交信息（如：修复登录按钮样式）"
# 跳过暂存区，直接提交工作区所有修改（仅限已跟踪文件）
git commit -am "提交信息"
```

4. 查看提交历史

```
# 查看完整提交历史（按 q 退出）
git log
# 简洁显示历史（每条记录一行）
git log --oneline
# 查看最近 n 条记录
git log -n 3（n 为数字，如 3 表示最近 3 条）
# 查看分支合并历史（图形化展示）
git log --graph --oneline
```

5. 撤销操作

```
# 撤销工作区修改（恢复到最近一次提交状态，未暂存的修改会丢失）
git checkout -- 文件名（如：git checkout -- lib/main.dart）
# 撤销暂存区修改（将文件从暂存区退回工作区）
git reset HEAD 文件名（如：git reset HEAD lib/main.dart）
# 回退到指定版本（慎用！会覆盖后续提交，commit_id 可通过 git log 查看）
git reset --hard 提交ID（如：git reset --hard a1b2c3d）
```

三、分支管理（新建、切换、删除等）

1. 查看分支

```
# 查看本地所有分支（* 表示当前分支）
git branch
# 查看远程所有分支（前缀 origin/）
git branch -r
# 查看本地 + 远程所有分支
git branch -a
```

2. 新建分支

```
# 新建本地分支（基于当前分支）
git branch 分支名（如：git branch feature/login）
# 新建并切换到该分支（常用）
git checkout -b 分支名（如：git checkout -b feature/login）
# 基于远程分支新建本地分支（同步远程分支内容）
git checkout -b 本地分支名 origin/远程分支名（如：git checkout -b dev origin/dev）
```

3. 切换分支

```
git checkout 分支名（如：git checkout main）
# 新版 Git 可用（更直观）
git switch 分支名（如：git switch main）
```

4. 删除分支

```
# 删除本地分支（需先切换到其他分支）
git branch -d 分支名（如：git branch -d feature/login）
# 强制删除未合并的本地分支（谨慎！）
git branch -D 分支名
# 删除远程分支
git push origin --delete 远程分支名（如：git push origin --delete feature/login）
```

四、同步与合并（远程交互、分支合并）

1. 远程仓库相关

```
# 查看远程仓库地址（origin 是远程仓库的默认别名）
git remote -v
# 关联远程仓库（首次使用时）
git remote add origin 远程仓库地址（如：git remote add origin https://github.com/xxx.git）
# 修改远程仓库地址
git remote set-url origin 新地址
```

2. 拉取远程代码（同步到本地）

```
# 拉取远程指定分支到当前本地分支（自动合并，等价于 git fetch + git merge）
git pull origin 远程分支名（如：git pull origin main）
# 拉取远程所有分支的最新信息（不合并，仅更新本地缓存）
git fetch origin
```

3. 推送本地代码到远程

```
# 推送本地分支到远程（首次推送需指定远程分支，后续直接 git push）
git push -u origin 本地分支名（如：git push -u origin main，-u 表示关联远程分支）
# 推送已关联的本地分支
```

```
git push
# 强制推送（覆盖远程分支，谨慎使用！适合纠正错误提交）
git push -f origin 分支名
```

4. 合并分支

```
# 切换到目标分支（如要将 feature 合并到 main，先切到 main）
git checkout main
# 合并指定分支到当前分支
git merge 分支名（如：git merge feature/login）
# 合并无关联历史的分支（解决 "refusing to merge unrelated histories" 错误）
git merge 分支名 --allow-unrelated-histories
```

5. 解决合并冲突

当合并提示冲突时：

1. 打开冲突文件，找到冲突标记（<<<<<<< HEAD 到 >>>>>>> 分支名 之间的内容）
2. 手动编辑保留需要的代码，删除冲突标记
3. 提交解决结果：

```
git add 冲突文件名
git commit -m "解决合并冲突：xxx"
```

五、其他实用命令

```
# 查看本地与远程分支的差异
git diff 本地分支名 origin/远程分支名（如：git diff main origin/main）
# 暂存当前工作区（临时切换分支时保存修改）
git stash
# 恢复最近一次暂存的内容
git stash pop
# 查看所有暂存记录
git stash list
# 删除暂存记录
git stash drop 暂存编号（如：git stash drop 0）
```

```
# 放弃本地所有未提交的修改（回到最近一次提交状态）
git reset --hard HEAD
# 清理工作区未跟踪的文件（如编译产物、临时文件）
git clean -f（删除文件）
git clean -fd（删除文件和目录）
```

以上命令覆盖了 Git 日常开发的绝大多数场景，核心流程可总结为：

克隆/初始化仓库 → 新建分支开发 → 暂存+提交本地修改 → 拉取远程同步 → 推送本地修改 → 合并分支。

熟练后可结合实际需求灵活组合使用。

（注：文档部分内容可能由 AI 生成）